PROPOSAL TUGAS AKHIR – MKXXXXXX

Automatic Code Generation: Transformasi Model BPMN menjadi Komponen React.js Berbasis *Rule-Based* dan Large Language Model.

Joseph Marcell Gultom

NRP 5025221279

Dosen Pembimbing

Dr. Kelly Rossa Sungkono, S.Kom., M.Kom.

NIP 1994201912088

Prof. Drs.Ec. Ir. Riyanarto Sarno, M.Sc., Ph.D.

NIP 195908031986011001

Program Studi Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2025

LEMBAR PENGESAHAN

AUTOMATIC CODE GENERATION*: TRANSFORMASI MODEL BPMN MENJADI KOMPONEN REACT.JS BERBASIS *RULE-BASED* DAN *LARGE LANGUAGE MODEL

PROPOSAL TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat

memperoleh gelar <Nama Gelar> pada

Program Studi S-1 Teknik Informatika

Departemen Teknik Informatika

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Oleh : **Joseph Marcell Gultom**

NRP. 5025221279

Disetujui oleh Tim Penguji Proposal Tugas Akhir :

1.	Dr. Kelly Rossa Sungkono, S.Kom., M.Kom.	Pembimbing
2.	Prof. Drs.Ec. Ir. Riyanarto Sarno, M.Sc., Ph.D.	Ko-pembimbing
3.	Nama dan gelar penguji	Penguji
4.	Nama dan gelar penguji	Penguji
5.	Nama dan gelar penguji	Penguji

SURABAYA

Bulan, Tahun

ABSTRAK

AUTOMATIC CODE GENERATION: TRANSFORMASI MODEL BPMN MENJADI KOMPONEN REACT.JS BERBASIS *RULE-BASED* DAN *LARGE LANGUAGE MODEL*

Nama Mahasiswa / NRP	**:**	**Joseph Marcell Gultom / 5025221279**
Departemen	**:**	**Teknik Informatika FTEIC – ITS**
Dosen Pembimbing	**:**	**Dr. Kelly Rossa Sungkono, S.Kom., M.Kom. NIP 1994201912088**
Dosen Co-Pembimbing	**:**	**Prof. Drs.Ec. Ir. Riyanarto Sarno, M.Sc., Ph.D. NIP 195908031986011001**

Abstrak

Business Process Model and Notation (BPMN) sebagai representasi model proses bisnis sering kali menghadapi ambiguitas semantik eksekusi karena kurangnya definisi formal, sehingga transformasi manual ke implementasi *frontend* seperti React.js berpotensi menimbulkan inkonsistensi logika dan kesenjangan antara analis bisnis dengan pengembang perangkat lunak. Ketiadaan standar elemen visual menyebabkan kesulitan dalam pembangkitan kode antarmuka yang akurat yang mana pendekatan *rule-based* saat ini masih terbatas pada level konseptual tanpa dukungan pemodelan data persisten. Tugas akhir ini mengusulkan kerangka kerja *hybrid* berbasis *rule-based* dan *Large Language Model* (LLM) untuk transformasi model BPMN menjadi proyek React.js secara otomatis. Pendekatan ini dirancang untuk menerjemahkan elemen BPMN seperti *flow objects*, *connecting objects*, *artifacts*, dan *swimlane* ke dalam struktur modular React.js yang mencakup halaman, navigasi, state management, serta integrasi API. Metodologi tugas akhir meliputi tiga tahap utama yaitu (1) persiapan dataset yang mencakup transformasi model BPMN ke *Intermediate Representation* 1 (IR1) dengan preservasi semantik, pengembangan algoritma transformasi IR1 ke IR2, dan pembuatan dataset, (2) pemrosesan LLM dengan teknik prompting LLM *Chain of Thought*, *Fine Tuning*, dan *Retrieval-Augmented Generation* pada model Qwen2.5-Coder:7b untuk meningkatkan kualitas *output* kode dan pengayaan dataset, serta (3) pengembangan *automatic code generator*. Metode evaluasi yang akan digunakan mencakup tiga aspek utama, yaitu (1) evaluasi kualitas kode yang dihasilkan melalui metrik objektif seperti *correctness*, *completeness*, dan *readability*, (2) perbandingan performa antara tiga varian pendekatan *hybrid*, yaitu *Chain of Thought + Baseline LLM*, *Chain of Thought + Fine Tuning*, dan *Chain of Thought + Fine Tuning + RAG*, serta (3) pengukuran efisiensi proses pengembangan melalui waktu generasi kode, jumlah iterasi manual yang diperlukan. Tugas akhir ini diharapkan memberikan kontribusi teoritis transformasi model visual BPMN ke kode eksekusi antarmuka modern seperti React.js.

Kata kunci: ***BPMN 2.0, React.js, Rule Based, LLM, Qwen2.5-Coder:7b, Fine Tuning, Chain of Thought, RAG, JSON Prompting, Model Driven Development, Intermediate Representation,*** **Generasi Kode Otomatis.**

ABSTRACT

AUTOMATIC CODE GENERATION: TRANSFORMATION OF BPMN MODELS INTO RULE-BASED AND LARGE LANGUAGE MODEL–DRIVEN REACT.JS COMPONENTS

Student Name / NRP	:	**Joseph Marcell Gultom / 5025221279**
Departement	:	**Informatics Engineering FTEIC – ITS**
Advisor	:	**Dr. Kelly Rossa Sungkono, S.Kom., M.Kom. NIP 1994201912088**
Co-Advisor	:	**Prof. Drs.Ec. Ir. Riyanarto Sarno, M.Sc., Ph.D. NIP 195908031986011001**

Abstract

Business Process Model and Notation (BPMN) as a representation of business process models, often encounters execution semantic ambiguity due to the lack of formal definitions, leading to potential logical inconsistencies and gaps between business analysts and software developers during manual transformation into frontend implementations such as React.js. The absence of visual element standards causes difficulties in generating accurate interface code, where current rule-based approaches remain limited to the conceptual level without support for persistent data modeling. This final project proposes a hybrid framework based on rule-based methods and Large Language Models (LLMs) for automatically transforming BPMN models into React.js projects. The approach is designed to translate BPMN elements such as flow objects, connecting objects, artifacts, and swimlanes into a modular React.js structure that includes pages, navigation, state management, and API integration. The methodology of this final project consists of three main stages: (1) dataset preparation, encompassing the transformation of BPMN models into Intermediate Representation 1 (IR1) with semantic preservation, development of algorithms for transforming IR1 into IR2, and dataset creation; (2) LLM processing using Chain of Thought prompting techniques, Fine Tuning, and Retrieval-Augmented Generation on the Qwen2.5-Coder:7B model to improve code output quality and dataset enrichment; and (3) development of an automatic code generator. The evaluation methods employed cover three main aspects: (1) assessment of generated code quality through objective metrics such as correctness, completeness, and readability; (2) performance comparison across three hybrid approach variants, namely Chain of Thought + Baseline LLM, Chain of Thought + Fine Tuning, and Chain of Thought + Fine Tuning + RAG; and (3) measurement of development process efficiency through code generation time and the number of required manual iterations. This final project is expected to provide a theoretical contribution to the transformation of visual BPMN models into modern interface execution code such as React.js.

Keywords: ***BPMN 2.0, React.js, Rule Based, LLM, Qwen2.5-Coder:7b, Fine Tuning, Chain of Thought, RAG, JSON Prompting, Model Driven Development, Intermediate Representation, Automatic Code Generation***.

DAFTAR ISI

LEMBAR PENGESAHAN i
ABSTRAK ii
ABSTRACT iii
DAFTAR ISI iv
DAFTAR GAMBAR vi
DAFTAR TABEL vii
DAFTAR SIMBOL viii
BAB 1 PENDAHULUAN 9
1.1 Latar Belakang 9
1.2 Rumusan Masalah 10
1.3 Batasan Masalah 10
1.4 Tujuan 10
1.5 Manfaat 10
BAB 2 TINJAUAN PUSTAKA 11
2.1 Hasil Penelitian Terdahulu 11
2.2 Dasar Teori 12
2.2.1 Business Process Model and Notation 12
2.2.2 React.js 13
2.2.3 Model Driven Engineering (MDE) 14
2.2.4 Rule Map 14
2.2.5 *Large Language Model* (LLM) 18
2.2.6 Chain of Thought 19
2.2.7 Retrieval Augmented Generation 19
2.2.8 Code Generation 20
BAB 3 METODOLOGI 21
3.1 Alur Metodologi Penelitian 21
3.1.1 Tahap 1: Persiapan Dataset 22
3.1.2 Tahap 2: Pemrosesan LLM 24
3.1.3 Tahap 3: Perancangan Automatic Code Generator 26
3.2 Metode yang digunakan 26
3.2.1 Representasi BPMN ke Intermediate Representation 26
3.2.2 Fine Tuning LLM 32

3.2.3 Retrieval-Augmented Generation 34
3.3 Lingkungan Perangkat Keras dan Perangkat Lunak 36
3.4 Jadwal Pengerjaan 36
BAB 4 Daftar Pustaka 37
LAMPIRAN 40

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SIMBOL

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Walaupun BPMN telah menjadi standar industri dalam pemodelan proses bisnis, notasi ini tidak menyediakan semantik eksekusi yang formal, sehingga implementasi ke sistem dilakukan dengan interpretasi manual yang berpotensi menyebabkan variasi logika eksekusi yang tidak konsisten pada perangkat lunak [1]. Ambiguitas teknis tersebut menciptakan kesenjangan komunikasi antara analis bisnis yang fokus pada proses bisnis dan pengembang sebagai teknis perangkat lunak [2]. Lebih jauh lagi, tantangan muncul pada pengembangan antarmuka pengguna karena BPMN tidak mendefinisikan standar elemen visual, sehingga proses transformasi ke kode antarmuka menjadi kurang akurat dan bergantung pada interpretasi subjektif pengembang [3]. Pendekatan transformasi ke antarmuka saat ini yang berbasis *rule-based* hanya mampu menghasilkan kode pada level konseptual dan belum diaplikasikan pada pembangkit kode antarmuka [4]. Selain itu, pemodelan data dan bahasa ekspresi BPMN yang menggunakan skema XML dan XPath menghambat pembangkitan kode karena tipe data terikat pada siklus hidup proses saja dan bukan struktur data aplikasi yang persisten [5].

Belum ditemukan penelitian yang secara spesifik mencoba merancang pembangkit kode untuk tranformasi transformasi bpmn ke kode react js berbasis *rule based* dan LLM, namun terdapat beberapa penelitian yang telah mengupayakan transformasi BPMN ke berbagai bentuk kode. Ouyang et al. dalam [6] menunjukkan pendekatan untuk menerjemahkan model BPMN yang berorientasi graf menjadi struktur blok kode *Business Process Execution Language* (BPEL) yang dapat dieksekusi pada layanan web. Sementara itu, pendekatan Debnath et al. pada [7] menunjukkan bahwa transformasi BPMN ke kode Java EE dapat dilakukan melalui berbagai tahap, dimulai dari pemetaan struktur kelas, aktivitas, dan relasi ke model *Unified Model Language* (UML), kemudian model UML diubah menjadi profil Java EE, dan terakhir model ditranformasikan menjadi komponen Java EE secara otomatis menggunakan *Meta Object Facility Script* (MOFScript). Cheon et al. pada [8] menunjukkan bahwa Large Language Models mampu menghasilkan kode dalam kerangka *Model Driven Development* (MDD) yang mana ini selaras dengan karakteristik BPMN sebagai artefak berbasis model, sehingga meningkatkan efisiensi proses pengembangan perangkat lunak. Niculescu et al. pada [9] telah melakukan upaya transformasi diagram BPMN ke komponen React.js, namun pendekatannya belum memanfaatkan sistem pembangkit kode terstruktur. Transformasi dilakukan melalui *prompt engineering* dengan menggabungkan unggahan citra diagram dan instruksi tekstual untuk menghasilkan beberapa jenis kode, salah satunya React.js. Metode ini memiliki keterbatasan karena LLM dalam hal ini ChatGPT tidak mampu menghasilkan keseluruhan hasil transformasi dalam satu percobaan sehingga diperlukan interaksi berulang melalui serangkaian prompt untuk memperoleh keluaran yang lengkap.

Berdasarkan permasalahan tersebut, tugas akhir ini mengusulkan pengembangan sistem pembangkit kode otomatis yang mentransformasikan model BPMN menjadi proyek React.js melalui pendekatan campuran berbasis *rule-based* dan *Large Language Model* (LLM). Pendekatan *rule-based* digunakan untuk menerjemahkan struktur BPMN, meliputi *flow objects*, *connecting objects*, *artifacts*, dan *swimlane* ke dalam representasi kerangka komponen React,

seperti struktur halaman, alur navigasi, data model, serta relasi antar komponen. Representasi terstruktur ini selanjutnya disintesis oleh LLM untuk menghasilkan kode React.js secara komprehensif mencakup antarmuka pengguna (JSX), *hooks* dan *state management*, logika bisnis, integrasi API, serta modularisasi kode. Pendekatan ini diharapkan mampu meningkatkan otomasi transformasi proses bisnis ke implementasi *frontend* dan sekaligus menawarkan kontribusi terhadap perluasan kajian model proses bisnis dalam konteks pengembangan perangkat lunak modern.

1.2 Rumusan Masalah

1. Bagaimana merancang aturan pemetaan model BPMN ke represetansi kerangka React.js
2. Bagaimana mengembangkan dan mengadaptasi LLM melalui teknik CoT, *fine tuning,* dan RAG, sehingga LLM dapat menghasilkan kode React.js dengan benar
3. Bagaimana membangun sistem pembangkit kode otomatis yang menerima model BPMN sebagai input dan menghasilkan proyek React.js yang lengkap, terstruktur, dan dapat dipelihara

1.3 Batasan Masalah

Tugas akhir ini memiliki batasan masalah, yaitu

1. Analisis terbatas pada cakupan elemen BPMN 2.0.
2. Data proses LLM berstruktur ChatML untuk *fine tuning* dan PDF untuk RAG.
3. Keluaran sistem merupakan aplikasi React.js sederhana berbasis Vite dan Typescript dengan CSR, *state management* menggunakan zustand, dan *routing* menggunakan *react router* tanpa ada *backend, database*, dan *deployment.*

1.4 Tujuan

Tujuan utama tugas akhir ini adalah:

1. Merumuskan dan mengimplementasikan aturan pemetaan model BPMN ke representasi kerangka React.js.
2. Mengembangkan model LLM yang paham dengan konteks semantik BPMN dan menghasilkan kode React.js dengan metode Cot, *fine tuning,* dan RAG.
3. Membangun sistem pembangkit kode yang menerima model BPMN sebagai masukan dan menghasilkan projek React.js lengkap yang bersih dan mudah dipelihara.

1.5 Manfaat

Berdasarkan tujuannya, manfaat tugas akhir ini adalah:

1. Memberikan kerangka pemetaan formal antar elemen BPMN dengan komponen React.js
2. Menyediakan model LLM yang sudah *fine-tuned* untuk interpretasi semantik BPMN.
3. Menghasilkan sistem pembangkit kode otomatis untuk React.js yang dapat mempercepat siklus pengembangan aplikasi

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Terdapat beberapa penelitian sebelumnya terkait transformasi BPMN ke bentuk kode eksekusi. Penjelasan mengenai penelitian – penelitian tersebut dapat dilihat pada tabel 2.1 Tabel Penelitian Terkait berikut:

Tabel 2.1 Tabel Penelitian Terkait

Judul paper dan referensi	Ringkasan	Perbandingan dengan Tugas Akhir
Business process model and notation (bpmn) version 2.0. [10]	Memetakan BPMN yang berorientasi graf ke dalam bahasa eksekusi BPEL. Elemen BPMN seperti *Process* dan *Service Task* diterjemahkan menjadi <process> dan <invoke>, sedangkan *Gateway* dipetakan ke struktur kontrol <if> dan <flow>. Struktur graf kompleks direpresentasikan menggunakan <flow> dan <links>, serta objek data BPMN dipetakan menjadi variabel BPEL.	Fokus pada orkestrasi layanan backend (BPEL), bukan pada antarmuka pengguna. Tidak ada pemetaan ke UI, state, atau komponen React.js. Pendekatan ini bersifat deterministik berbasis aturan formal, tanpa pemanfaatan LLM, RAG, atau pembangkitan kode frontend modern
Transformation of Business Process Models BPMN 2.0 into components of the Java Business Platform [7]	Menggunakan pendekatan *Model Driven Architecture* (MDA). BPMN dipetakan ke UML (PIM), kemudian diperkaya dengan profil Java EE (PSM), dan akhirnya diubah menjadi kode Java EE (EJB, JPA, *service interface*) menggunakan MOFScript.	Berorientasi pada backend enterprise berbasis Java EE, bukan frontend React.js. Transformasi bersifat model-to-model dan model-to-text klasik
LLMs as Code Generators for Model-Driven Development [8]	Mengkaji penggunaan LLM untuk menghasilkan kode dari artefak berbasis model seperti BPMN dalam kerangka *Model Driven Development* (MDD). Namun hasil masih tidak konsisten, bergantung pada kualitas prompt, dan tidak tervalidasi otomatis.	Tidak ditargetkan khusus pada React.js dan tidak terintegrasi dengan pembangkit kode otomatis
Model-Driven Development Using LLMs: The Case of ChatGPT. [9]	Menggunakan LLM, yaitu ChatGPT dengan masukan berupa gambar diagram BPMN dan prompt teks untuk menghasilkan berbagai jenis kode, termasuk React.js. Prosesnya iteratif,	Tidak terintegrasi dengan sistem pembangkit kode terstruktur dan tidak menerakan *rule based*

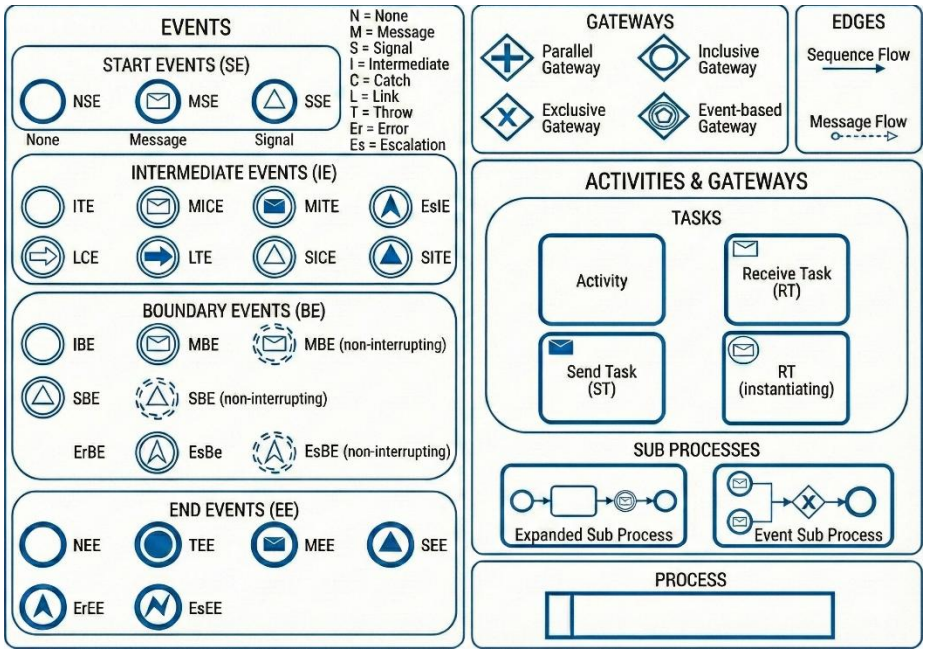
	memerlukan banyak prompt, dan tidak sepenuhnya otomatis.	*mapping*, *intermediate representation* (IR).

2.2 Dasar Teori

2.2.1 Business Process Model and Notation

Business Process Model and Notation (BPMN) 2.0 merupakan standar internasional yang dikembangkan oleh Object Management Group (OMG) untuk mendefinisikan representasi grafis proses bisnis. BPMN menyediakan elemen elemen utama seperti events, activities, gateways, dan flows untuk menggambarkan alur aktivitas secara formal namun mudah dipahami oleh pengguna teknis maupun non-teknis. Versi 2.0 memperkenalkan peningkatan penting berupa dukungan terhadap eksekusi model dan penyelarasan dengan format XML (XML Schema Definition) agar kompatibel dengan sistem otomatisasi proses bisnis (Business Process Management Systems). BPMN banyak digunakan dalam industri maupun akademik karena sifatnya yang standar, *interoperabel*, dan mudah divisualisasikan [11].

Kräuter et al. pada [12] menguraikan bahwa BPMN memiliki struktur elemen yang bersifat hierarkis yang dimulai dari Collaboration sebagai kontainer tingkat atas yang merepresentasikan interaksi antar *participant*. Setiap *participant* mereferensikan sebuah Process. Pada tingkat proses, BPMN mendefinisikan FlowElements yang membentuk perilaku dan alur kontrol proses. Elemen ini terbagi menjadi FlowNode dan SequenceFlow. FlowNode memodelkan perubahan status dalam proses dan diklasifikasikan menjadi tiga katogeri, yaitu Activities yang merepresentasikan pekerjaan yang dilakukan, termasuk Sub Process yang berfungsi untuk modularisasi dan peningkatan keterbacaan, Gateways yang mengelola percabangan, penggabungan, dan pengendalian alur seperti pada Parallel Gateway untuk eksekusi paralel dan Exclusive Gateway untuk pemilihan jalur tunggal berdasarkan kondisi, serta Events yang memodelkan kejadian selama eksekusi proses yang mencakup Start Event dan End Event sebagai penanda awal dan akhir, Message Event untuk komunikasi satu ke satu entitas, Signal Event untuk penyiaran global yang bisa diterima seluruh partisipan, Link Event sebagai penghubung alur dalam diagram yang kompleks, dan *Terminate End Event* yang mengakhiri proses tanpa menunggu penyelesaian aktivitas lain. Selain SequenceFlow yang berperan sebagai penghubung antar FlowNode dalam satu proses, BPMN juga menyediakan *Message Flow* yang digunakan untuk memodelkan komunikasi antar *Participant* pada Tingkat *Collaboration*.



Gambar 2.1 Pengelompokkan Elemen BPMN disalin dari [12]

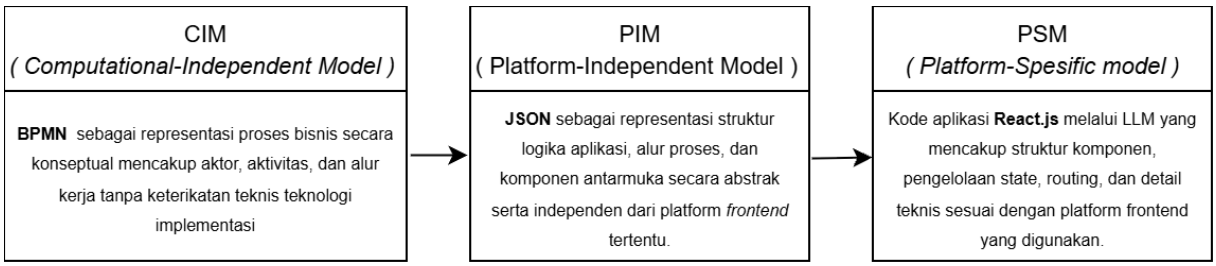
2.2.2 React.js

React.js merupakan pustaka JavaScript *open-source* yang dikembangkan untuk membangun antarmuka pengguna (UI) khususnya untuk aplikasi *web single-page*. Pustaka ini memungkinkan pengembang untuk membuat aplikasi web berskala besar yang dapat menggunakan dan mengubah data secara dinamis tanpa perlu memuat ulang halaman. Keunggulan utama React.js terletak pada implementasi *Virtual* DOM yang meningkatkan performa aplikasi secara keseluruhan. React menggunakan algoritma *diffing* yang cerdas untuk memperbarui *node* DOM yang diperlukan saja, sementara elemen lainnya tetap tidak berubah, Hal ini meminimalisir manipulasi DOM yang mahal dan meningkatkan responsivitas aplikasi. Arsitektur berbasis komponen React memungkinkan pengembang untuk membangun UI yang dapat digunakan kembali, sehingga menyederhanakan proses pengembangan dan pemeliharaan aplikasi [13].

Fitur-fitur penting React.js mencakup arsitektur berbasis komponen, Virtual DOM, metode rekonsiliasi, dan keunggulan React Hooks [14]. React mengadopsi konsep *unidirectional data flow* yang memastikan data mengalir dalam satu arah di seluruh aplikasi, sehingga state dan model aplikasi terkontrol dengan lebih baik [13]. JSX (JavaScript XML) digunakan sebagai ekstensi sintaks, memungkinkan pengembang untuk menulis struktur UI menggunakan sintaks yang menyerupai HTML di dalam kode JavaScript, membuat kode lebih intuitif dan mudah dipahami. Penelitian terbaru menunjukkan bahwa React.js telah menjadi pilihan utama dalam pengembangan web modern karena performanya yang superior dibandingkan framework lain seperti Angular dan Vue, terutama dalam menangani aplikasi dengan frekuensi perubahan data yang tinggi dan berdataset besar [15]. React Hooks yang diperkenalkan dalam versi terbaru, menjelaskan cara baru untuk mengelola *state* dan *lifecycle methods* dalam *functional components*, sehingga meningkatkan keterbacaan kode dan mengurangi kompleksitas [13].

2.2.3 Model Driven Engineering (MDE)

Model-Driven Engineering (MDE) adalah metodologi pengembangan perangkat lunak dengan basis model sebagai artefak utama yang menggerakan proses pengembangan [16]. Tujuan utama MDE terbagi menjadi tiga tingkat, yaitu transformasi dan penyempurnaan model dalam bentuk *computation-independent model* (CIM) ke bentuk *platform-independent model* (PIM), dan ke dalam bentuk akhir *platform-specific model* (PSM). Pada tingkat CIM, kebutuhan, konteks, dan tujuan model direpresentasikan secara abstrak tanpa terikat aspek komputasi atau teknologi implementasi tertentu. BPMN relevan untuk digunakan pada tingkat CIM karena sifatnya yang menggambarkan alur kerja, aktor, dan tujuan bisnis secara umum tanpa terkait teknis teknologi tertentu. Kemudian, tingkat selanjutnya, yaitu PIM mendeskripsikan perilaku dan struktur aplikasi secara independent tanpa kaitan suatu teknologi berdasarkan pemetaan dari informasi yang disajikan di CIM. Elemen – elemen BPMN dimodelkan secara abstrak sebagai representasi interaksi antar pengguna, transisi *state*, dan alur logika dalam bentuk JSON. Tingkat terakhir, yaitu PSM mendefinisikan dan mewujudkan perilaku dan struktur aplikasi secara spesifik pada platform tujuan yang mana dalam konteks tugas akhir ini, yaitu React.js. Pada tingkat ini, hasil pemetaan PIM diimplementasikan ke dalam antarmuka termasuk struktur komponen, *state management*, *event handling,* dan elemen sintaksis JSX, sehingga model proses bisnis dapat ditransformasi menjadi aplikasi *frontend* yang dapat dieksekusi. [17]



Gambar 2.2 Pemodelan bertingkat dalam kerangka MDE

2.2.4 Rule Map

Aturan pemetaan (*rule mapping*) didefinisikan sebagai seperangkat kaidah formal yang menetapkan hubungan antara elemen-elemen BPMN dan representasi struktural aplikasi *frontend* dalam kerangka React.js. Aturan ini bersifat deterministik, sehingga setiap elemen BPMN yang sama akan selalu menghasilkan representasi yang konsisten pada tahap transformasi berikutnya. Dengan pendekatan ini, ambiguitas interpretasi model proses bisnis dapat diminimalkan, serta keterlacakan antara model BPMN dan artefak implementasi dapat terjaga. Representasi ini memuat informasi mengenai jenis aktivitas, alur kontrol, peran pengguna, serta mekanisme interaksi antar proses secara abstrak. Dengan demikian, rule mapping berperan sebagai fondasi utama dalam proses transformasi model, khususnya pada tahap peralihan dari CIM ke PIM.

Pemetaan elemen menghasilkan *Intermediate Representation* (IR) yang terdiri dari dua tingkat. Tingkat pertamaa merupakan pemetaan pertama yang menangkap strutktur esensial proses bisnis secara umum tanpa terikat teknis React.js. Pada level ini, elemen BPMN seperti *pool*, *lane*, *task*, *event*, *gateway*, serta *sequence flow* direpresentasikan secara abstrak sebagai entitas proses, peran, aktivitas, alur kontrol, dan dependensi data.

```
{
 "collaboration": {
  "id": "",
  "name": "",
  "pools": [
   {
    "id": "",
    "name": "",
    "processRef": ""
   }
  ]
 },
 "lanes": [
  {
   "id": "",
   "name": "",
   "poolId": ""
  }
 ],
 "tasks": [
  {
   "id": "",
   "name": "",
   "type": "",
   "role": "",
   "interactionHint": "",
   "stateModel": [],
   "asyncFlag": false,
   "incoming": [],
   "outgoing": []
  }
 ],
 "events": [
  {
   "id": "",
   "name": "",
   "eventType": "",
   "trigger": "",
   "incoming": [],
   "outgoing": []
  }
 ],
 "gateways": [
```

```
    {
      "id": "",
      "name": "",
      "gatewayType": "",
      "controlHint": "",
      "incoming": [],
      "outgoing": []
    }
  ],
  "sequenceFlows": [
    {
      "id": "",
      "source": "",
      "target": "",
      "condition": ""
    }
  ],
  "messageFlows": [
    {
      "id": "",
      "sourcePool": "",
      "targetPool": "",
      "messageType": ""
    }
  ],
  "stateSchema": {}
}
```

Setelah pemetaan awal BPMN berhasil diperoleh, pemetaan berlanjut ke tingkat dua yang mana mengintegrasikan struktur BPMN ke format JSON *prompting* untuk *content generation* di LLM.

```
{
  "task": "generate",
  "description": "Generate a complete React.js application codebase based on a structured
process and UI model derived from a business workflow.",

  "input": {
    "context": {
      "application_goal": "Implement a React.js frontend that follows the given process flow
and role-based interaction model",
      "architecture_style": "modular",
      "coding_convention": "clean, maintainable, and idiomatic React"
    },

    "model": {
      "project": {
        "name": "",
        "entryComponent": "",
        "routingStrategy": "client-side"
```

```
    },

    "modules": [
      {
        "name": "",
        "pages": [
          {
            "name": "",
            "route": "",
            "uiType": ""
          }
        ],
        "components": [
          {
            "name": "",
            "componentType": ""
          }
        ],
        "services": [
          {
            "name": "",
            "async": false
          }
        ]
      }
    ],

    "routing": [
      {
        "path": "",
        "page": ""
      }
    ],

    "state": {
      "global": [],
      "local": []
    },

    "flowControl": []
  }
},

"schema": {
  "fields": [
    {
      "name": "files",
      "type": "array",
      "description": "List of generated React.js source files with file paths and code content."
```

```
      }
    ]
  },

  "constraints": {
    "output_format": "JSON",
    "max_length":
  }
}
```

Dengan format masukan terstruktur tersebut, LLM menghasilkan keluaran berupa artefak kode aplikasi React.js yang direpresentasikan dalam format JSON. Artefak ini selanjutnya dimanfaatkan oleh *code generator* untuk mengekstraksi dan menempatkan setiap berkas ke dalam jalur direktori yang sesuai secara deterministik.

```
{
  "files": [
    {
      "path": "",
      "content": ""”
    }
  ]
}
```

2.2.5 *Large Language Model* (LLM)

Large Language Model (LLM) adalah model *deep learning* yang dilatih dengan data berjumlah besar untuk memahami dan menghasilkan bahasa alami serta jenis konten lainnya untuk melakukan berbagai tugas [18]. Zhao et al. dalam [19] menjelaskan bahwa model bahasa mengalami evolusi dari statistical language models ke neural language models, kemudian ke pre-trained language models (PLMs), dan akhirnya mencapai large language models ketika parameter scale melebihi tingkat tertentu. Kemampuan LLM dalam memahami dan menghasilkan general-purpose language diperoleh melalui pelatihan miliaran parameter model pada data teks dalam jumlah masif, sebagaimana diprediksi oleh aturan scaling laws [20].

2.2.5.1 Transformer

LLM modern dibangun diatas arsitektur *transformer* yang merupakan model transduksi urutan berbasis *attention* yang menggantikan arsitektur terdahulu, yaitu *recurrent* dan *convolutional* dengan mekanisme *self-attention* [21]. Mekanisme *attention* memungkinkan model untuk memeriksa relevansi berbagai bagian dari input secara paralel, sehingga dependensi jarak jauh dalam teks dapat ditangkap dengan lebih efektif [22]. Model Transformer terdiri dari encoder dan decoder yang masing-masing terdiri dari *multiple layers* dengan *multi-head attention* dan *feed-forward networks*, sehingga model mampu mempelajari representasi kontekstual yang kaya [23]. Pada tugas akhir ini, transformer digunakan sebagai pustaka untuk

segala proses LLM seperti memperoleh *pretrained model*, *fine tuning*, *tokenization* dan penggunaan LoRa.

2.2.5.2 LLM *Fine-tuning*

Fine tuning adalah metode pelatihan model kecerdasan buatan untuk menghasilkan LLM yang spesifik dan terspesialisasi pada bidang tertentu dengan cara melatih model awal dengan himpunan dataset yang sudah dilabeli [24]. Dalam upaya tranformasi BPMN ke kode, *Fine tuning* relevan karena mampu menghasilkan LLM yang terspesialisasi dengan biaya pelatihan yang murah dibandingkan *pre training* [25]. *Fine tuning* sebuah LLM terbagi menjadi empat proses besar, yaitu *chat templates, supervised fine-tuning* (SFT), *low rank adaption* (LoRA), dan evaluasi.

Chat templates merupakan kerangka format struktural untuk mengarahkan model LLM dalam menghasilkan keluaran yang konsisten, terkontrol, dan selaras dengan kebutuhan. *Template* tersebut secara eksplisit mendefinisikan tujuan tugas (task), deskripsi, konteks input, skema keluaran, serta batasan (constraints), sehingga berfungsi sebagai spesifikasi formal yang menjembatani kebutuhan pengguna dengan proses generasi teks oleh model. Dengan pendekatan ini, interaksi tidak lagi bersifat percakapan bebas, melainkan menyerupai kontrak semantik antara pengguna dan sistem AI. Pada tugas akhir ini, format *chat template* yang untuk model Qwen2.5-Coder:7B adalah ChatML.

2.2.6 Chain of Thought

Chain-of-Thought (CoT) *prompting* merupakan teknik *prompting* yang menambahkan rangkaian langkah penalaran eksplisit dalam bentuk bahasa alami di antara input dan output sehingga model LLM dapat melakukan dekomposisi masalah secara bertahap sebelum menghasilkan jawaban akhir. Pendekatan ini menggabungkan dua gagasan utama, yaitu penggunaan *natural language rationales* sebagai representasi langkah penalaran dan kemampuan *few-shot in-context learning* pada large language models, sehingga model tidak hanya memetakan *input* ke *output* secara langsung, tetapi juga mengeksekusi proses inferensi internal yang lebih terstruktur. Secara empiris, CoT prompting terbukti secara signifikan meningkatkan kinerja pada tugas-tugas yang menuntut penalaran aritmatika, *commonsense*, dan simbolik, khususnya pada model LLM berskala sangat besar (≥100B parameter). Selain meningkatkan akurasi, CoT juga menyediakan jendela interpretabilitas karena langkah - langkah penalaran yang dihasilkan dapat dianalisis untuk memahami atau menelusuri kesalahan model, sehingga menjadikannya tidak hanya efektif secara performa tetapi juga bernilai secara metodologis dalam penelitian sistem AI berbasis penalaran. [26]

2.2.7 Retrieval Augmented Generation

Retrieval-Augmented Generation (RAG) merupakan pendekatan *hybrid* dalam pemodelan bahasa yang mengintegrasikan memori parametrik dari model bahasa neural dengan memori non-parametrik berbasis mekanisme *retrieval untuk mendukung tugas-tugas NLP yang bersifat knowledge intensive*. Dalam RAG, sebuah *neural retriever* digunakan untuk mengambil dokumen relevan dari sebuah indeks pengetahuan eksternal berdasarkan masukan pengguna. Kemudian informasi tersebut dikondisikan bersama masukan ke dalam model *sequence-to-sequence* yang bertindak sebagai generator. Secara probabilistik, dokumen yang diambil diperlakukan sebagai variabel laten yang dimarginalkan untuk menghasilkan distribusi keluaran, baik pada tingkat urutan penuh (*RAG-Sequence*) maupun per token (*RAG-Token*). Arsitektur

ini memungkinkan model untuk menggabungkan pengetahuan eksplisit yang dapat diperbarui dengan kemampuan generatif model bahasa, sehingga mengurangi halusinasi, meningkatkan ketepatan faktual, serta memberikan fleksibilitas dalam pembaruan pengetahuan tanpa perlu melatih ulang seluruh model. [27]

2.2.8 Code Generation

Dalam [28], dijelaskan bahwa salah satu kemampuan LLM, yaitu *natural-language-to-code* adalah membangkitkan kode, termasuk React.js berdasarkan masukan berupa deskripsi bahasa alami. Secara formal, bahasa alami direpresentasikan sebagai x. Dengan diberikannya x, penggunaan LLM dengan parameter model θ untuk menghasilkan berupa solusi kode y dapat dinotasikan sebagai Pθ (y | x). Munculnya kemampuan baru LLM, yaitu *in-context-learning,* telah mendorong penambahan beberapa contoh atau *exemplars* ke dalam deskripsi bahasa alami x, sehingga performa LLM meningkat, baik dalam membangkitkan kode ataupun membatasi format kode yang dihasilkan. Sebuah himpunan tetap *exemplars* dinotasikan sebagai {(xi , yi)}M i = 1 , oleh karena itu formula pembangkitan kode *zero shot* atau *few shot* menggunakan LLM dijelaskan dalam tabel 2.3 Penjelasan formula LLM dalam *code generation* berikut:

$$P_{\theta(y|x)} \Rightarrow P_{\theta\left(y\middle|prompt\left(x,\{(x_i,y_i)\}_{\{i=1\}}^{k}\right)\right)}, k \in \{0,1,\dots,M\}$$

Tabel 2.3 Penjelasan formula LLM dalam *code generation*

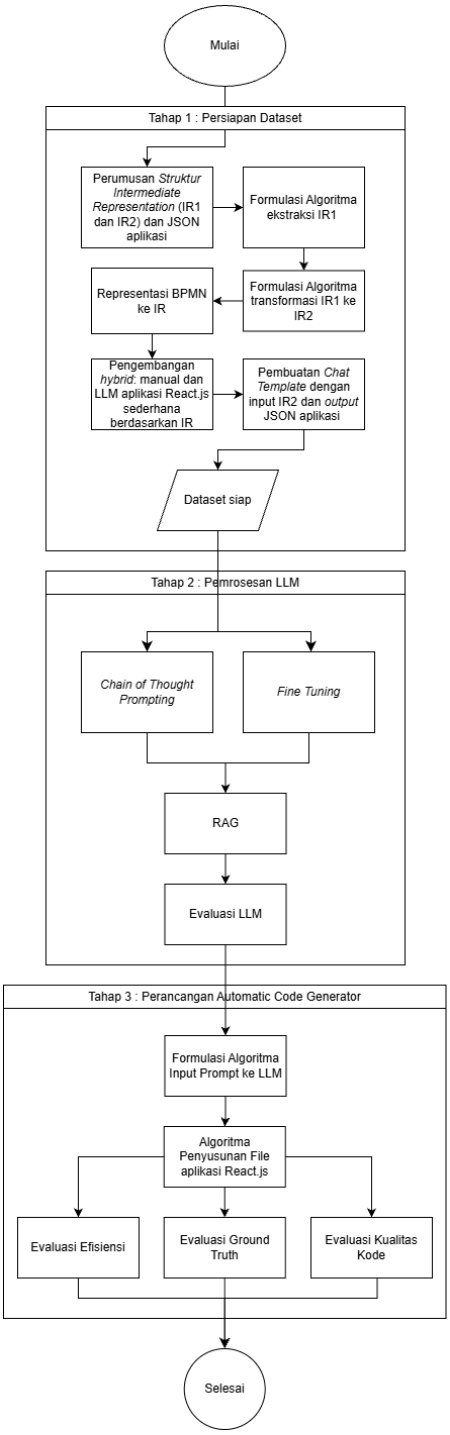
Tabel Penjelasan	
$prompt(x,\{(x_i,y_i)\}_{\{i=1\}}^{k}$	Representasi string masukan yang berupa deskripsi bahasa alami
$x,\{(x_i,y_i)\}_{\{i=1\}}^{k}$	Himpunan *exemplars* k dipilih acak dari {(xi , yi)}M i = 1

BAB 3 METODOLOGI

Bab 3 menjelaskan metodologi yang digunakan dalam tugas akhir dalam mengembangkan pembangkit kode otomatis untuk transformasi model BPMN ke kode React.js.

3.1 Alur Metodologi Penelitian

Pada tugas akhir ini, terdapat tiga tahap besar yang dilakukan untuk pengembangan pembangkit kode otomatis untuk transformasi model BPMN ke kode React.js. Tahap 1 merupakan persiapan dataset yang didalamnya mencakup perumusan struktur *intermediate representation* (IR), formulasi algoritma ekstraksi IR1, formulasi algoritma transformasi IR1 ke IR2, representasi BPMN ke IR, pengembangan *hybrid*: manual dan LLM aplikasi React.js sederhana berdasarkan IR, dan pembuatan *chat template* dengan input IR2 dan output JSON aplikasi. Tahap 2 merupakan pemrosesan LLM yang mencakup beberapa langkah, yaitu *chain of thought prompting*, *fine tuning,* RAG, dan evaluasi LLM. Tahap 3, yaitu tahap terakhir mencakup formulasi algoritma input *prompt* ke LLM, algoritma penyusunan file aplikasi react.js, evaluasi efisiensi, evaluasi *ground truth*, dan evaluasi kualitas kode.



Gambar 3.1 Desain Metode Penelitian

3.1.1 Tahap 1: Persiapan Dataset

Tahap pertama dari dalam metodologi penelitian ini adalah menyiapkan dataset untuk *fine* tuning LLM dan RAG. LLM memerlukan dataset berbentuk *chat template* untuk merespon dengan benar terhadap instruksi tertentu dan belum ditemukannya dataset transformasi BPMN ke aplikasi React.js, sehingga diperlukan pembuatan dataset secara mandiri. Proses ini dimulai dengan merumuskan struktur *Intermediate Representation (IR)* dan *JSON* aplikasi. Struktur IR1 menguraikan model BPMN dengan ekstraksi seluruh elemen dan relasinya termasuk *events,*

activities, gateways, pool, lanes, dan *artifacts* dalam bentuk JSON. Kemudian, IR2 memetakan model BPMN beserta elemen – elemennya dalam konteks aplikasi React.js dan mengintegrasikannya ke dalam struktur JSON *prompting* yang objektifnya adalah implementasi antarmuka React.js. Secara umum, LLM akan merespon IR2 dengan menghasilkan JSON aplikasi yang berisikan semua *file path* dan isi filenya. Kemudian, langkah selanjutnya adalah formulasi algoritma modular untuk memperoleh IR1 dari model BPMN melalui serangkaian prosedur ekstraksi terpisah yang dirancang secara spesifik untuk setiap jenis elemen. Algoritma - algoritma ini mencakup ekstraksi *pool* dan *lanes*, aktivitas (tasks), kejadian (events), gerbang keputusan (gateways), alur urutan *internal* (*sequence flows*), alur pesan eksternal (*message flows*), serta *data objects* dan *artifacts* lainnya. Setiap algoritma melakukan *parsing* dokumen XML BPMN menjadi struktur pohon, diikuti dengan pencarian dan pengumpulan elemen - elemen relevan beserta atribut utamanya, sehingga menghasilkan representasi data yang terstruktur dan independent. Pendekatan modular ini memungkinkan pemrosesan yang lebih terfokus, mudah dipelihara, serta mendukung integrasi lebih lanjut untuk generasi kode aplikasi.

Formulasi selanjutnya dikerjakan untuk transformasi IR1 ke IR2 dengan inisialisasi kerangka IR2 sesuai skema target, termasuk pengaturan metadata tugas, konteks aplikasi, dan kendala output. Selanjutnya, informasi proyek seperti nama aplikasi diambil dari elemen proses utama BPMN, sementara modul-modul aplikasi dibentuk berdasarkan pool yang merepresentasikan pemisahan peran dalam proses bisnis. Halaman - halaman React dibuat dari *tasks* dengan pemetaan nama, rute URL yang jelas, dan tipe antarmuka sesuai karakteristik task tersebut. Kemudian, dibuat tabel *routing* dan inferensi *state* global dari alur pesan serta variabel proses, sehingga menghasilkan model IR2 yang lengkap, modular, dan sesuai untuk generasi kode antarmuka pada LLM. Langkah selanjutnya dalam penelitian ini adalah pembuatan dataset pasangan *Intermediate Representation 2* (IR2) dan kode aplikasi React.js dalam jumlah yang signifikan melalui pendekatan *hybrid* yang mengintegrasikan pengembangan manual dan pemanfaatan *Large Language Model* (LLM). Pada tahap awal, sejumlah aplikasi React.js dibangun secara manual sebagai *benchmark* dengan merujuk secara ketat pada spesifikasi yang terkandung dalam IR2, sehingga menghasilkan referensi *ground truth* yang berkualitas tinggi dan dapat diandalkan untuk evaluasi selanjutnya. Selanjutnya, dataset diperluas secara substansial dengan memperoleh variasi IR2 dari beragam file BPMN yang mencakup kasus proses bisnis dengan kompleksitas dan domain yang berbeda. Aplikasi React.js yang telah dikembangkan secara manual tersebut kemudian dimanfaatkan sebagai *retrieval model* dalam mekanisme RAG pada model LLM berkinerja tinggi seperti GPT dan Grok. Proses generasi kode dilakukan dengan memasukkan IR2 sebagai input ke LLM, sehingga dihasilkan output berupa kode aplikasi React.js. Setiap output yang dihasilkan dievaluasi secara ketat berdasarkan metrik *correctness*, *completeness*, dan *readability* dan jika memenuhi ambang batas kualitas, maka pasangan IR2 dan kode React.js tersebut dimasukkan ke dalam dataset berbentuk *chat template*. Pendekatan iteratif ini diharapkan dapat menghasilkan dataset berskala besar yang representatif serta mendukung peningkatan performa sistem pembangkit kode otomatis dalam kerangka kerja *hybrid* yang diusulkan.

3.1.2 Tahap 2: Pemrosesan LLM

Pada tahap kedua ini LLM disiapkan sebagai alat generasi kode dengan menerapkan Chain of Thought (CoT), Fine tuning, dan RAG, sehingga respon yang dihasilkan memiliki akurasi tinggi dan benar. Pertama tama, CoT diterapkan pada model LLM dalam tugas akhir ini untuk memandu proses penalaran secara bertahap dan logis dalam menghasilkan kode React.js dari Intermediate Representation 2 (IR2). Pendekatan ini melibatkan desain *prompt* yang memerintahkan model untuk memecah tugas kompleks menjadi langkah-langkah sekuensial, seperti analisis modul berdasarkan *swimlane* BPMN, pembentukan struktur *routing* berbasis peran, definisi *state management global*, serta adaptasi pola komponen dari *benchmark* aplikasi React.js yang dibangun secara manual sebagai *ground truth*. Dengan merujuk pada *benchmark* manual tersebut yang mencakup implementasi modular dengan *role-based access*, penggunaan *zustand* untuk *process state*, dan integrasi interaksi antar halaman seperti tombol aksi untuk simulasi message flow, teknik CoT memastikan bahwa output LLM tidak hanya mengikuti spesifikasi IR2, tetapi juga menjaga konsistensi dengan *best practices coding* React.js yang telah divalidasi secara manual. Hasilnya, teknik CoT ini meningkatkan akurasi dan *readability* kode yang dihasilkan, sekaligus mengurangi halusinasi model dengan menjadikan benchmark sebagai referensi eksplisit dalam rantai penalaran. Berikut adalah struktur CoT yang akan digunakan dalam projek tugas akhir ini

Chain of Thought Prompt

Model Input

Q: Transform this IR2.JSON into React.js app

{ "task": "generate", "input": { "context": { ... } }, "schema": { ... }, "constraints": { ... } }

A: The input is an IR2 specification for generating a React.js app. First, identify the project name and setup client-side routing. Next, map modules to folders like src/modules/customer. Then, for each page, determine uiType and create corresponding components. Infer global state from state.global. Build routing with role guards. Finally, generate support files.

{ "files": [{ "path": "src/App.tsx", "content": "..." }] }

Q: Transform another IR2.JSON into React.js app

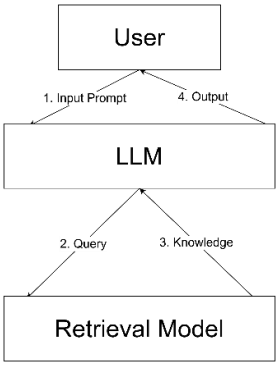
Model Output

A: { "files": [{ "path": "src/App.tsx", "content": "..." }] }

Tahap selanjutnya adalah *fine tuning* pada LLM model Qwen2.5-Coder:7B untuk mengadaptasi model dasar agar lebih terspesialisasi terhadap pola transformasi spesifik dari Intermediate Representation 2 (IR2) ke kode React.js yang modular dan selaras dengan alur

proses bisnis BPMN. Proses fine tuning menggunakan Low-Rank Adaptation (LoRA) dengan dataset pasangan IR2 dan kode React.js dalam format *ChatML.* Parameter yang digunakan adalah *learning rate* 2e-4, *epochs* 3, dan rank LoRA 16 untuk menjaga efisiensi komputasi sambil meningkatkan akurasi output, sehingga model mampu menghasilkan kode yang lebih readable, dan bebas halusinasi dibandingkan varian baseline. Tahap terakhir dari pemrosesan LLM adalah RAG yang berfungsi meningkatkan pengetahuan LLM dengan menyediakan pengetahuan eksternal terkait BPMN, React.js, dan transformasinya setelah proses *fine tuning*. Basis pengetahuan dalam implementasi RAG ini dibangun dari tiga kelompok sumber data utama. Pertama, dataset internal berupa *benchmark* struktur proyek React.js yang mencakup seluruh file dari folder proyek yang telah divalidasi secara manual dan dataset pasangan IR2 dan kode React.js . Kedua, dokumentasi resmi BPMN 2.0 yang diunduh dari situs Object Management Group (OMG) yang menjelaskan BPMN secara keseluruhan. Ketiga, dokumentasi resmi React.js yang berisikan segala pengetahuan dasar untuk membangun aplikasi React.js.

Tahap berikutnya adalah *preprocessing* dan *chunking*, yaitu memecah dokumen berukuran besar menjadi unit-unit kecil yang tetap bermakna dengan panjang sekitar 500 – 1200 token. Proses ini dapat dilakukan menggunakan metode seperti *recursive character splitting* atau semantic. Setiap potongan teks tersebut kemudian dikonversi menjadi vektor numerik melalui proses *embedding* menggunakan model seperti *SentenceTransformer all-MiniLM-L6-v2* atau *bge-small-en-v1.5*, yang merepresentasikan makna semantik dalam ruang berdimensi tinggi (misalnya 384 atau 512 dimensi). Selanjutnya, vektor hasil *embedding* disimpan dan diindeks dalam basis data vector seperti ChromaDB, FAISS, atau Pinecone. Struktur ini memungkinkan pencarian berbasis kemiripan semantik, seperti *cosine similarity* atau *dot product*, dengan performa yang sangat efisien. Output dari fase ini adalah sebuah retrieval model yang siap digunakan, yaitu basis data vektor untuk memperoleh pengetahuan yang paling relevan secara semantik terhadap suatu kueri. Pada fase generasi kode, representasi IR2 atau CoT terlebih dahulu diubah menjadi vektor kueri. Basis data vektor kemudian melakukan pencarian top-k dan hasilnya disisipkan ke dalam prompt akhir yang diberikan kepada LLM.



Gambar 3.2 Sistem RAG pada LLM

3.1.3 Tahap 3: Perancangan Automatic Code Generator

Tahap ketiga dalam penelitian ini berfokus pada perancangan dan implementasi *Automatic Code Generator* yang berfungsi sebagai komponen akhir dari *pipeline* untuk mengubah Intermediate Representation 2 (IR2) menjadi aplikasi React.js yang lengkap, modular, dan siap dijalankan. *Generator* ini dirancang sebagai sistem otomatis berbasis LLM yang sudah diproses dengan pendekatan berlapis yang mencakup pembuatan prompt terstruktur, generasi kode melalui model Qwen2.5-Coder-7B yang dijalankan secara lokal menggunakan Ollama, serta *post processing* otomatis untuk menghasilkan struktur direktori proyek React.js yang sesuai dengan best practices modern.

Proses dimulai dengan pembuatan prompt yang sangat eksplisit dan ketat, di mana IR2 dalam format JSON (yang telah dibangun dari hasil parsing BPMN dan pemetaan ke struktur React seperti nama proyek, daftar halaman, route, serta tipe UI) disisipkan langsung ke dalam instruksi. Prompt memuat peran model sebagai expert React.js developer, batasan teknologi (functional components, React Hooks, React Router v6), struktur folder yang diharapkan (src/pages untuk halaman utama), serta aturan khusus untuk setiap tipe task (form untuk userTask, confirmation untuk serviceTask/sendTask/receiveTask). Output LLM dibatasi secara ketat hanya berupa objek JSON valid terstruktur tanpa teks tambahan, penjelasan, atau markdown, untuk memastikan mempermudah *parsing* otomatis.

Setelah LLM menghasilkan respon, dilakukan tahap *post-processing* menggunakan skrip Python yang mencakup pembersihan output, seperti menghapus kode fence. Tahap akhir menampilkan instruksi pengguna untuk menjalankan aplikasi dengan perintah standar, seperti cd [nama_proyek], *npm install*, dan *npm run dev*, sehingga pengguna dapat langsung melihat dan menguji aplikasi React.js yang telah dihasilkan. Dengan pendekatan ini, sistem mampu menghasilkan aplikasi frontend yang modular, bersih, dan sesuai dengan standar React.js modern (Vite + TypeScript) dalam waktu singkat, sekaligus memanfaatkan hasil *fine tuning* LoRA dan RAG pada tahap sebelumnya untuk memastikan akurasi transformasi, konsistensi arsitektur, serta pengurangan halusinasi secara signifikan. Tahap ini menjadi puncak dari keseluruhan *pipeline* penelitian, mengubah representasi abstrak proses bisnis menjadi aplikasi web interaktif yang fungsional secara sepenuhnya otomatis.

3.2 Metode yang digunakan

3.2.1 Representasi BPMN ke Intermediate Representation

Representasi model BPMN ke *intermediate representation* merupakan metode awal dalam pengerjaan tugas akhir ini. Proses ini terdiri dari dua tahapan utama, yaitu BPMN ke IR1 dan IR1 ke IR2. Pada tahapan pertama, file BPMN diekstraksi dengan tugas fungsi guna menangkap esensi utama file BPMN, yaitu *pool* dan lanes, *activities, events, gateways, sequence flow, message flow,* dan *data objects/artifacts.* Proses ekstraksi ini sebatas merangkum model BPMN untuk transformasi lebih lanjut, sehingga tidak ada terminologi React.js yang digunakan. Berikut adalah semua algoritma pseudocode yang digunakan pada tahap pertama :

Pseudocode 3.1 Algoritma ekstraksi *Pool* dan *Lanes*

Input: bpmn_file (BPMN 2.0 XML)
Output: pools_and_lanes (dictionaries of pools and lanes)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. collaboration_section ← Find_Collaboration_Section(root_node)
3. Initialize pools_and_lanes ← []
4. IF collaboration_section IS NOT None THEN // Extract pools (participants)
5. FOR EACH participant IN Find_Participants(collaboration_section):
6. pool_info ← { "id": participant.id, "name": participant.name, "type": "pool", "linked_process": participant.processRef }
7. Append pool_info to pools_and_lanes
8. END FOR
9. END IF
10. FOR EACH lane_set IN Find_All_Lane_Sets(root_node):
11. FOR EACH lane IN Find_Lanes(lane_set):
12. lane_info ← { "id": lane.id, "name": lane.name, "type": "lane", "contained_elements": lane.flowNodeRefs }
13. Append lane_info to pools_and_lanes
14. END FOR
15. END FOR
16. RETURN pools_and_lanes

Pseudocode 3.2 Algoritma ekstraksi *Activities*

Input: bpmn_file (BPMN 2.0 XML)
Output: activities (dictionaries of tasks and other activities)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. Initialize activities ← []
3. activity_types ← ['userTask', 'serviceTask', 'task', 'sendTask', 'receiveTask', 'manualTask', 'businessRuleTask', 'scriptTask']
4. FOR EACH type_name IN activity_types:
5. FOR EACH element IN Find_Elements_By_Type(root_node, type_name):
6. activity ← { "id": element.id, "name": element.name, "type": type_name, "incoming_connections": [], "outgoing_connections": [] }
7. Append activity to activities
8. END FOR
9. END FOR
10. RETURN activities

Pseudocode 3.3 Algoritma ekstraksi *Events*

Input: bpmn_file (BPMN 2.0 XML)
Output: events (dictionaries of BPMN events)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. Initialize events ← []
3. event_types ← ['startEvent', 'endEvent', 'intermediateCatchEvent', 'intermediateThrowEvent', 'boundaryEvent']
4. FOR EACH type_name IN event_types:
5. FOR EACH element IN Find_Elements_By_Type(root_node, type_name):

6. event ← { "id": element.id, "name": element.name, "event_kind": type_name, "incoming_connections": [], "outgoing_connections": [] }
7. Append event to events
8. END FOR
9. END FOR
10. RETURN events

Pseudocode 3.4 Algoritma ekstraksi *Gateways*

Input: bpmn_file (BPMN 2.0 XML)
Output: gateways (dictionaries of decision points)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. Initialize gateways ← []
3. gateway_types ← ['exclusiveGateway', 'parallelGateway', 'inclusiveGateway', 'complexGateway', 'eventBasedGateway']
4. FOR EACH type_name IN gateway_types:
5. FOR EACH element IN Find_Elements_By_Type(root_node, type_name):
6. gateway ← { "id": element.id, "name": element.name, "decision_kind": Extract_Type_Name(type_name), "incoming_connections": [], "outgoing_connections": [] }
7. Append gateway to gateways
8. END FOR
9. END FOR
10. RETURN gateways

Pseudocode 3.5 Algoritma ekstraksi *Sequence Flow*

Input: bpmn_file (BPMN 2.0 XML)
Output: sequence_flows (dictionaries of internal connections)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. Initialize sequence_flows ← []
3. FOR EACH flow IN Find_Sequence_Flows(root_node):
4. flow_info ← { "id": flow.id, "from": flow.sourceRef, "to": flow.targetRef, "condition": flow.name }
5. Append flow_info to sequence_flows
6. END FOR
7. RETURN sequence_flows

Pseudocode 3.6 Algoritma ekstraksi *Message Flow*

Input: bpmn_file (BPMN 2.0 XML)
Output: message_flows (dictionaries of inter-pool communications)

1. root_node ← Parse_BPMN_XML(bpmn_model)
2. collaboration_section ← Find_Collaboration_Section(root_node)
3. Initialize message_flows ← []
4. IF collaboration_section IS NOT None THEN

```
5. FOR EACH message IN Find_Message_Flows(collaboration_section):
6. message_info ← { "id": message.id, "from_participant": message.sourceRef,
   "to_participant": message.targetRef, "message_kind": message.name }
7. Append message_info to message_flows
8. END FOR
9. END IF
10. RETURN message_flows
```

Pseudocode 3.7 Algoritma ekstraksi *Data Objects* dan *Artifacts*

```
Input: bpmn_file (BPMN 2.0 XML)
Output: data_and_artifacts (dictionaries of data objects and artifacts)
1. root_node ← Parse_BPMN_XML(bpmn_model)
2. Initialize data_and_artifacts ← []
3. // Extract data objects and references
4. FOR EACH obj IN Find_Data_Objects(root_node):
5. obj_info ← { "id": obj.id, "name": obj.name, "type": "dataObject", "is_collection":
   obj.isCollection }
6. Append obj_info to data_and_artifacts
7. END FOR
8. FOR EACH ref IN Find_Data_Object_References(root_node):
9. ref_info ← { "id": ref.id, "name": ref.name, "type": "dataObjectReference",
   "linked_object": ref.dataObjectRef }
10. Append ref_info to data_and_artifacts
11. END FOR
12. // Extract artifacts (annotations, groups, associations)
13. artifact_types ← ['textAnnotation', 'group', 'association']
14. FOR EACH type_name IN artifact_types:
15. FOR EACH element IN Find_Elements_By_Type(root_node, type_name):
16. artifact_info ← { "id": element.id, "name": element.name, "type": type_name }
17. Append artifact_info to data_and_artifacts
18. END FOR
19. END FOR
20. RETURN data_and_artifacts
```

Pada tahap kedua, IR1 sebagai representasi BPMN ringkas ditransformasikan lebih spesifik ke representasi aplikasi React.js yang lebih kontekstual dalam bentuk IR2. Keterangan transformasi elemen BPMN ke React.js dijabarkan pada tabel 3.1 berikut :

Tabel 3.1 Representasi elemen BPMN dalam React.js

Elemen BPMN	**Hasil Transformasi di React.js**
Process	Nama proyek aplikasi React.js
Collaboration / Pools	Modul – modul terpisah

Lanes	Submodul dan Role based route
UserTask	Halaman React dengan uiType: "form"
ServiceTask / ScriptTask	Halaman dengan uiType: "confirmation" atau "standard"
SendTask / ReceiveTask	Halaman confirmation atau display
StartEvent	Halaman pertama (route "/")
EndEvent	Halaman akhir proses
SequenceFlow	Navigasi linear antar halaman
MessageFlow	Dipetakan ke state global untuk sinkronisasi data antar modul/peran
Exclusive Gateway	Logika conditional rendering atau protected route berdasarkan kondisi
Parallel Gateway	Beberapa halaman/tab yang bisa diakses secara paralel atau urutan fleksibel
Inclusive Gateway	Beberapa halaman bisa dilewati tergantung kondisi
Data Objects / Variables	Variabel proses diubah menjadi global state

Untuk membentuk IR2, digunakan sebuah algoritma yang diawali dengan menginisialisasi struktur IR2 sebagai objek kosong sesuai dengan skema target yang telah ditentukan. Struktur ini kemudian diisi dengan metadata proyek, meliputi *task* bertipe *generate*, deskripsi tujuan sistem, spesifikasi keluaran berupa *array* file, dan *constraint* bahwa seluruh keluaran harus mengikuti format JSON. Setelah itu, bagian konteks aplikasi ditetapkan dengan tujuan membangun aplikasi frontend React.js yang merefleksikan alur proses bisnis dan pola interaksi berbasis peran, dengan mengadopsi arsitektur modular dan konvensi pengkodean React yang bersih serta mudah dipelihara.

Nama proyek diturunkan dari elemen utama BPMN, baik berupa process maupun collaboration, sementara entry component ditetapkan sebagai *App* dengan mekanisme client-side routing. Setiap pool atau kolaborasi pada BPMN diterjemahkan menjadi modul terpisah dengan penamaan yang telah dinormalisasi. Di dalam setiap modul, task BPMN dipetakan menjadi halaman React, dengan konvensi nama berakhiran *Page*, rute yang dibentuk dari nama modul dan *slug* task, serta atribut uiType yang disesuaikan dengan tipe task, misalnya *form* untuk *userTask* atau *standard* untuk *sendTask*. Berdasarkan pemetaan tersebut, sistem kemudian membangkitkan daftar routing yang lengkap.

Selanjutnya, seluruh modul yang terbentuk dimasukkan ke dalam struktur IR2. *State* global diturunkan dari *message flows* dan *data objects* BPMN, sedangkan state lokal dibiarkan kosong untuk diisi pada tahap generasi kode. Pola alur kontrol seperti linear, bercabang, atau paralel

diidentifikasi berdasarkan *gateway* dan *sequence flow*. Setelah seluruh elemen tersebut terintegrasi, IR2 yang telah terisi secara komprehensif dikembalikan sebagai masukan terstruktur bagi LLM untuk menghasilkan implementasi kode React.js yang sesungguhnya. Berikut adalah pseudocode 3.8 algoritma transformasi IR1 ke IR2.

Pseudocode 3.8 Algoritma transformasi IR1 ke IR2

Input: ir1
Output: ir2

```
1. Initialize ir2 ← empty structure matching the target schema
2. // Set project metadata
3. ir2['task'] ← "generate"
4. ir2['description'] ← "Generate a complete React.js application codebase based on a
   structured process and UI model derived from a business workflow."
5. ir2['schema']['fields'][0] ← { "name": "files", "type": "array", "description": "List of
   generated React.js source files with file paths and code content." }
6. ir2['constraints']['output_format'] ← "JSON"
7. // Fill context section
8. ir2['input']['context']['application_goal'] ← "Implement a React.js frontend that
   follows the given process flow and role-based interaction model"
9. ir2['input']['context']['architecture_style'] ← "modular"
10. ir2['input']['context']['coding_convention'] ← "clean, maintainable, and idiomatic
    React"
11. // Set project details
12. project_name ← Derive_Project_Name_From_IR1(ir1) // e.g., from main process
    name or collaboration name
13. ir2['input']['model']['project']['name'] ← project_name
14. ir2['input']['model']['project']['entryComponent'] ← "App"
15. ir2['input']['model']['project']['routingStrategy'] ← "client-side"
16. // Extract and create modules based on pools
17. Initialize modules_list ← []
18. FOR EACH pool IN ir1['collaboration']['pools']:
19. module_name ← Normalize_Name(pool['name']) // e.g., "Customer" → "customer"
20. module_entry ← { "name": module_name, "pages": [], "components": [],
    "services": [] }
21. Append module_entry to modules_list
22. END FOR
23. // Populate pages from tasks within each pool
24. FOR EACH task IN ir1['tasks']:
25. Determine owning_pool from task location in lanes or flow context
26. module_name ← Find_Corresponding_Module(owning_pool, modules_list)
27. page_name ← Capitalize_and_Normalize(task['name']) + "Page"
28. route_path ← "/" + module_name + "/" + Slugify(task['name'])
29. ui_type ← Map_Task_Type_to_UI(task['type']) // e.g., userTask → "form",
    sendTask → "standard"
30. page_entry ← { "name": page_name, "route": route_path, "uiType": ui_type }
31. Append page_entry to corresponding module's pages in modules_list
32. END FOR
33. // Generate routing table
34. Initialize routing_list ← []
```

35. FOR EACH module IN modules_list:
36. FOR EACH page IN module['pages']:
37. routing_entry ← { "path": page['route'], "page": page['name'] }
38. Append routing_entry to routing_list
39. END FOR
40. END FOR
41. ir2['input']['model']['routing'] ← routing_list
42. // Assign modules to IR2
43. ir2['input']['model']['modules'] ← modules_list
44. // Infer global state from message flows and process variables
45. global_state_vars ← Extract_Global_Variables_From_MessageFlows_and_DataObjects(ir1)
46. ir2['input']['model']['state']['global'] ← global_state_vars
47. ir2['input']['model']['state']['local'] ← []
48. // Infer flow control (gateways, parallel branches)
49. flow_controls ← Analyze_Gateways_and_Parallel_Flows(ir1['gateways'], ir1['sequenceFlows'])
50. ir2['input']['model']['flowControl'] ← flow_controls
51. RETURN ir2

3.2.2 Fine Tuning LLM

Tahap *fine tuning* pada Qwen2.5-Coder-7B dilakukan untuk menyesuaikan model dasar agar memiliki kompetensi yang lebih terfokus dalam mentransformasikan IR2 menjadi kode React.js yang modular, dan sesuai dengan alur proses bisnis BPMN. Proses adaptasi ini menerapkan teknik Low-Rank Adaptation (LoRA) untuk efisiensi komputasi yang mana hanya sebagian kecil parameter model yang diperbarui sehingga kebutuhan sumber daya dapat ditekan tanpa mengorbankan kualitas keluaran. Dataset disusun dalam bentuk pasangan IR2 dan kode React.js berformat *ChatML*, dengan konfigurasi pelatihan meliputi learning rate 2×10^{-4}, *batch size* 4, 3 *epoch*, serta *parameter* LoRA *rank* 16 dan *alpha* 32. Kombinasi ini dipilih untuk mencapai keseimbangan antara laju konvergensi dan pengendalian risiko overfitting. Implementasi pelatihan memanfaatkan *Hugging Face Transformers* dan PEFT yang terintegrasi dengan Unsloth, sehingga proses *fine tuning* dapat dipercepat hingga dua kali lipat pada satu GPU. Berikut adalah beberapa kode tahapan fine tuning pada Qwen2.5-Coder-7B.

Kode 3.1 Persiapan Model dan Dataset

```
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments
from peft import get_peft_model, LoraConfig
from datasets import load_dataset

model_name = "Qwen/Qwen2.5-Coder-7B"
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto",
load_in_4bit=True)
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
# Load dataset pasangan IR2 → Kode React.js dalam format ChatML
dataset = load_dataset("json", data_files={"train": "fine_tune_data.json"})
def preprocess_function(examples):
return tokenizer(examples["input"], examples["output"], truncation=True,
max_length=1024)

tokenized_dataset = dataset.map(preprocess_function, batched=True)
```

Kode 3.2 Konfigurasi LoRA dan Trainer

```
lora_config = LoraConfig(
    r=16,  # Rank LoRA
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],  # Modul transformer yang diadaptasi
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

training_args = TrainingArguments(
    output_dir="./lora_qwen_coder",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    fp16=True,  # Mixed precision untuk hemat VRAM
    save_steps=500,
    logging_steps=100,
    optim="adamw_torch"
)

from trl import SFTTrainer
trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    tokenizer=tokenizer,
    peft_config=lora_config
)
```

Kode 3.3 Pelatihan dan Penyimpanan Model

```
trainer.train()

# Simpan adapter LoRA
```

```
model.save_pretrained("lora_adapter")

tokenizer.save_pretrained("lora_adapter")

# Merge LoRA dengan model base (opsional untuk deployment)

merged_model = model.merge_and_unload()

merged_model.save_pretrained("fine_tuned_qwen_coder_7b")
```

3.2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) digunakan sebagai mekanisme penguatan eksternal untuk melengkapi keterbatasan pengetahuan internal model setelah proses fine-tuning. Seluruh sumber pengetahuan eksternal untuk LLM diproses melalui tahap *chunking* dengan ukuran sekitar 800–1200 token dan *overlap* 150 token, kemudian direpresentasikan dalam bentuk vektor menggunakan model *SentenceTransformer all-MiniLM-L6-v2* dan diindeks ke dalam ChromaDB sebagai vector penyimpanan yang dilengkapi metadata seperti sumber, bagian dokumen, dan lokasi berkas. Ketika IR2 diberikan sebagai masukan, mekanisme CoT digunakan untuk membentuk kueri retrieval yang spesifik, sehingga sistem dapat mengembalikan 6–10 chunk dengan relevansi tertinggi. Potongan - potongan ini selanjutnya disisipkan ke dalam prompt akhir LLM dengan instruksi eksplisit untuk merujuk pada golden template, dokumentasi BPMN, dan React.js, sehingga kode yang dihasilkan tetap konsisten dengan struktur direktori ideal, semantik proses bisnis, serta memenuhi standar industri kode React.js. Berikut adalah beberapa kode tahapan dalam implementasi RAG :

Kode 3.4 Persiapan Indexing Basis Pengetahuan

```
import chromadb
from chromadb.utils import embedding_functions
import os

client = chromadb.PersistentClient(path="./rag_knowledge_db")
embedding_fn =
embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-
MiniLM-L6-v2")

collection = client.get_or_create_collection(
    name="react_bpmn_rag",
    embedding_function=embedding_fn
)

# Fungsi untuk index folder golden template
def index_golden_template(project_path):
    documents, metadatas, ids = [], [], []
    for root, _, files in os.walk(project_path):
        for file in files:
            if file.endswith((".tsx", ".ts", ".jsx", ".js", ".json", ".md")):
```

```
            path = os.path.join(root, file)
            rel_path = os.path.relpath(path, project_path)
            with open(path, "r", encoding="utf-8") as f:
                content = f.read()
            documents.append(content)
            metadatas.append({"source": "golden_template", "path": rel_path})
            ids.append(rel_path.replace("/", "_").replace(".", "_"))
    if documents:
        collection.add(documents=documents, metadatas=metadatas, ids=ids)
        print(f"✓ {len(documents)} file golden template diindeks")

# Fungsi untuk index PDF/docs BPMN & React
def index_document(text_content, source_name, section):
    chunk_size = 1000
    chunks = [text_content[i:i+chunk_size] for i in range(0, len(text_content), chunk_size)]
    documents = chunks
    metadatas = [{"source": source_name, "section": section} for _ in chunks]
    ids = [f"{source_name}_{i}" for i in range(len(chunks))]
    collection.add(documents=documents, metadatas=metadatas, ids=ids)

index_golden_template("./my-golden-react-app")
# index_document(bpmn_spec_text, "bpmn_official", "Gateways")  # dari PDF BPMN
# index_document(react_docs_text, "react_official", "Hooks")   # dari react.dev
```

Kode 3.5 *Retrieval* dan *Augment Prompt* saat generasi Kode

```
def retrieve_relevant_chunks(query, top_k=6):
    results = collection.query(
        query_texts=[query],
        n_results=top_k,
        include=["documents", "metadatas"]
    )
    contexts = []
    for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
        contexts.append(f"[{meta['source']} - {meta.get('path', meta.get('section',
"))}]\n{doc}")
    return "\n\n".join(contexts)

# penggunaan dalam prompt
ir2_json = ...  # IR2 Anda
query = f"Best React patterns for {ir2_json['input']['model']['modules'][0]['name']} module
with role-based routing and Zustand"
retrieved = retrieve_relevant_chunks(query)

full_prompt = f"""
You are expert React developer. Use ONLY the following retrieved knowledge:

{retrieved}
```

```
Generate code strictly based on this IR2:
{json.dumps(ir2_json['input']['model'], indent=2)}

Follow golden template structure, BPMN semantics, and React official docs.
Output ONLY JSON: {{"files": [{{"path": "...", "content": "..."}}]}}
"""
# Kirim ke LLM
```

3.3 Lingkungan Perangkat Keras dan Perangkat Lunak

Pengerjaan tugas akhir ini direncanakan menggunakan dua lingkungan komputer yang berbeda, yaitu laptop pribadi dan komputer Laboratorium Penelitian MCI. Tujuan dari pengerjaan di dua perangkat yang berbeda adalah penyesuaian kebutuhan kerja pada tahap – tahap tertentu, seperti eksperimen pemrosesan LLM yang membutuhkan sumber daya komputasi tinggi. Penjelasan terkait lingkungan pengerjaan tugas akhir dijabarkan pada tabel 3.2 Perangkat Keras dan Perangkat Lunak berikut:

Tabel 3.2 Perangkat Keras dan Perangkat Lunak

Kategori	Komponen	Spesifikasi
Perangkat Keras	Model Sistem	Victus by HP Laptop 16-d0xxx
	Processor 11th Gen Intel(R)	11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz (16 CPUs), ~2.3GHz
	Memori	16384MB
Perangkat Lunak	Bahasa Pemograman	Python 3.11
	Virtual Enviroment	Conda
	IDE	VsCode

3.4 Jadwal Pengerjaan

Berikut adalah jadwal pelaksaaan pengerjaan tugas akhir pada tabel 3.3 berikut:

No	Kegiatan	Minggu ke-															
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	Studi Literatur																
2	Persiapan Dataset																
3	Perancangan dan Implementasi Sistem																
4	Eksperimen dan Evaluasi																
5	Penulisan Buku Tugas Akhir																

BAB 4 Daftar Pustaka

[1] G. Zeisler, T. T. Braunauer, A. Fleischmann and R. Singer, "A Step Towards a Universal Method for Modeling and Implementing Cross-Organizational Business Processes.," *arXiv (Cornell University),* 2024.

[2] G. Huerta, J. A. Boubaker and H. Mili, "A business process re-engineering approach to transform BPMN models to software artifacts.," *International Conference on E-Technologies. Cham: Springer International Publishing,* 2017.

[3] Eduardo, E. Díaz, J. I. Panach, S. Rueda and J. Vanderdonckt, "An empirical study of rules for mapping BPMN models to graphical user interfaces," *Multimedia Tools and Applications,* 2021.

[4] E. Diaz, J. I. Panach, S. Rueda and O. Pastor, "Towards a method to generate GUI prototypes from BPMN," *In 2018 12th International Conference on Research Challenges in Information Science (RCIS), pp. 1-12. IEEE,* 2018.

[5] I. Drave, J. Michael, E. Müller, B. Rumpe and S. Varga, "Model-Driven engineering of Process-Aware information systems," *SN Computer Science,* 2022.

[6] C. Ouyang, M. Dumas, A. H. T. Hofstede and W. M. V. d. Aalst, "From BPMN process models to BPEL web services.," *In 2006 IEEE International Conference on Web Services (ICWS'06), pp. 285-292. IEEE,* 2006.

[7] N. Debnath, C. A. Martinez, F. Zorzan, D. Riesco and G. Montejano, "Transformation of Business Process Models BPMN 2.0 into components of the Java Business Platform," *IEEE 10th International Conference on Industrial Informatics. IEEE,* 2012.

[8] Y. Cheon, "LLMs as Code Generators for Model-Driven Development.," 2025.

[9] V. Niculescu, M.-C. Chisăliță-Crețu, C.-C. Osman and A. Sterca, "Model-Driven Development Using LLMs: The Case of ChatGPT.," 2025.

[10] OMG, "Business process model and notation (bpmn) version 2.0.," *Object Management Group,* 2013.

[11] M. Geiger, S. Harrer, J. Lenhard and G. Wirtz, "BPMN 2.0: The state of support and implementation," *Future Generation Computer Systems,* vol. 80, 2017.

[12] T. Kräuter, A. Rutle, H. König and Y. Lamo, "A higher-order transformation approach to the formalization and analysis of BPMN using graph transformation systems," *Logical Methods in Computer Science,* 2024.

[13] P. Rawat and A. N. Mahajan, "ReactJS: A Modern Web Development Framework," *International Journal of Innovative Science and Research Technology,* vol. 5, no. 11, 2020.

[14] P. Keshari, P. Maurya, P. Kumar and A. Katiyar, "Web development using ReactJS," pp. 1571-1575, 2023.

[15] A. S. Rajput, H. P. Singh, G. Bang, S. Joshi and T. Patidar, "Comparing Spring Boot and ReactJS with other web development frameworks: a study," *Lecture notes in networks and systems,* pp. 149-160, 2024.

[16] J. Hutchinson, J. Whittle and M. Rouncefield, "Model-driven engineering practices in industry: Social, organizational and managerial factors that lead to success or failure," *Science of Computer Programming,* pp. 144-161, 2013.

[17] M. Brambilla, J. Cabot and M. Wimmer, Model-Driven software engineering in practice, 2017.

[18] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes and A. Mian, "A comprehensive overview of large language models," *ACM Transactions on Intelligent Systems and Technology,* pp. 1-72, 2025.

[19] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie and J.-R. Wen, "A survey of large language models," *arXiv (Cornell University),* 2023.

[20] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain and J. Gao, "Large Language Models: a survey," *arXiv (Cornell University),* 2024.

[21] A. Vashwani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," *31st Conference on Neural Information Processing Systems (NIPS 2017),* 2017.

[22] D. Bahdanau, K. Cho and Y. Bengio∗, "Neural Machine Translation by Jointly Learning to Align and Translate," *ICLR 2015,* 2015.

[23] T. Lin, Y. Wang, X. Liu and X. Qiu, "A Survey of Transformers," *AI Open,* pp. 111-132, 2022.

[24] K. W. Church, Z. Chen and Y. Ma, "Emerging trends: A gentle introduction to fine-tuning," *Natural Language Engineering,* 2021.

[25] Y. Xia, J. Kim, Y. Chen, H. Ye, S. Kundu, C. Hao and N. Talati, "Understanding the performance and estimating the cost of LLM Fine-Tuning," *arXiv (Cornell University),* 2024.

[26] Google Research, Brain Team, "Chain-of-Thought Prompting Elicits Reasoning," *arXiv (Cornell University),* 2023.

[27] Facebook AI Research; ‡University College London; ?New York University;, "Retrieval-Augmented Generation for," *UCL Discovery (University College London),* vol. 33, pp. 9459-9474, 2020.

[28] J. Jiang, F. Wang, J. Shen, S. Kim and S. Kim, "A survey on large language models for code Generation," *arXiv (Cornell University),* 2024.

[29] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed and M. A. Sattar, "Industrial applications of large language models," *Scientific Reports,* 2025.

[30] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu and D. Amodei, "Scaling laws for neural language models," *arXiv (Cornell University),* 2020.

LAMPIRAN